

BdOI Monthly Round 1 Solutions: Problem Groupchat

Problem Information

Problem Author: Aritro Sarkar

Problem Preparation: Aritro Sarkar, Nabeul Islam

Editorial: Aritro Sarkar, Md Sazid Hasan

Subtask by subtask solutions

Preliminary Observation:

Let S_i be the last message seen by the i -th person. Let m be the current total number of messages sent.

Let a simple simulation of events be:

For every query of type 1, the number of sent messages increase by 1. Formally, $m := m + 1$.

For every query of type 2, friend f reads till the very last message m . Formally, $S_f := m$.

For every query of type 3, we are tasked with finding the number of indices i , such that, $S_i \geq s$.

Since $N \cdot Q \leq 2.5 \cdot 10^{11}$, it is clear that a standard simulation of the events will result in a Time Limit Exceeded error. However, it will yield partial scores.

Subtask 1: $N = 1$

This subtask is quite easy. A standard simulation of events will suffice. However, there is an even easier solution. For each query of type 1, m , the total number of messages sent so far will increase by 1. Formally, $m := m + 1$. For every query of type 2, we will set S_1 to m . Formally, $S_1 := m$. For every query of type 3,

$$\text{The answer } \mathbf{A} = \begin{cases} 1 & \text{if } S_1 \geq s, \\ 0 & \text{otherwise} \end{cases}$$

Maintaining this simple simulation with a loop will suffice.

Subtask 2: $N, Q \leq 5000$

For this subtask, a simulation of the events that occur will yield points. The simulation goes as follows:

Type 1: $m := m + 1$.

Type 2: $S_f := m$

Type 3:

$$\text{The answer } A = \sum_{i=1}^n \begin{cases} 1 & \text{if } S_1 \geq s, \\ 0 & \text{otherwise} \end{cases}$$

Subtask 3: All type 3 queries appear after all other queries

This subtask is special. Once all the type 1 and type 2 queries have taken place, only after, we will have to answer type 3 queries and S will not change again. Each type 3 queries ask us how many elements of S are larger or equal to the input s . We can solve this sub-problem using an ordered set, but an even easier solution is using difference array and prefix sum techniques. Here's how: We will do everything as the same for types 1 and 2 queries. Let D be the difference array. D_i is going to help us calculate how many people have seen the i -th message. Initially, $D_i = 0$ for all indices. For every person from 1 to n , D_{1,S_i} should be increased by 1. For every person i , $D_i := D_i + 1$ and $D_{S_i+1} := D_{S_i+1} - 1$.

Afterwards, we will perform prefix sum on D .

Formally, let $P_i = \sum_{j=1}^i D_j$. Now, P_i is the number of people who have read the i -th message. Our answer for a type 3 query is P_s .

Full solution description

Looking back at the standard simulation, the only reason it won't give full scores is that because it's time complexity is $O(N \cdot Q)$. Our simulation does type 1 and type 2 queries in $O(1)$, but the type 3 queries take $O(N)$ complexity. An ordered set or a segment tree can fully solve the problem. details are as follows:

Ordered Set: For every type 1 query, we assign it a timestamp which is equal to the number of current queries and insert it to a vector which contains the assigned timestamp of every sent message, let it be M . For every type 2 query, if $S_f \neq 0$, then we erase S_f from the ordered set. After, we update S_f . Now, we insert S_f to the ordered set.

For every type 3 query, the answer is `os.size() - os.order_of_key( $M_{s-1}$ )`.

Segment Tree or Fenwick Tree: Let each of the tree indices denote the timestamp. Now, similarly to the ordered set solution, when we are given a type 2 query if $S_f \neq 0$, we decrease 1 from the S_f th index of the tree, and after updating S_f , we increase the new S_f th index by 1.

Now, for query of type 3, the answer is the sum of the range $[M, R]$, where R denotes the last index of the tree.

Both of these solution achieves a Time Complexity of $O(Q \cdot \log_2 n)$, which will suffice for this problem's constraints.